

Agents, MCP, and the Path from Workflow to Agent

What Is an Agent?

The term "agent" has become one of the most overused words in AI, but Anthropic's engineering team offers a clear and useful definition. In their words, **agents** are systems where "LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks." This stands in contrast to **workflows**, which are systems where "LLMs and tools are orchestrated through predefined code paths."

The distinction comes down to **who controls the path**. In a workflow, the human developer decides the sequence of steps ahead of time. The LLM executes each step but doesn't choose what comes next. In an agent, the LLM itself decides which tools to call, in what order, and whether to continue or stop based on the results it receives.

Anthropic emphasizes that this isn't about one being "better" than the other—it's about choosing the right tool for the job. They recommend "finding the simplest solution possible, and only increasing complexity when needed." Workflows offer "predictability and consistency for well-defined tasks," while agents are "the better option when flexibility and model-driven decision-making are needed at scale."

What Is MCP?

The Model Context Protocol (MCP) is an open standard that acts as "a USB-C port for AI applications." Just as USB-C provides a standardized way to connect devices, MCP provides a standardized way to connect AI applications like Claude to external systems—data sources, tools, and workflows.

MCP defines three core primitives:

- **Tools:** Model-controlled functions that an LLM can actively call to take actions—like making API requests, querying databases, or writing files.

- Resources:** Application-controlled data that the client attaches to provide context—like file contents or database records.
- Prompts:** Pre-built instruction templates that guide language model interactions.

The key insight is that **who initiates the interaction** determines the primitive. With tools, the model decides to act. With resources, the application decides what context to provide. This clean separation makes MCP powerful: it lets developers build once and integrate across clients like Claude, ChatGPT, and Cursor, all of which support the protocol.

My FL-04 Workflow: A Workflow, Not an Agent

Based on the level 2 DFD I created for "Manage Workflow/Phase Progress," my FL-04 pipeline is clearly a **workflow** by Anthropic's definition. The diagram shows three predetermined steps: (1) Receive Advance Request, (2) Validate Gate (Checklist Status), and (3) Update Phase State. The path is fixed—a project manager requests a phase advance, the system checks checklist status, and then updates the phase state. The LLM (if present) isn't deciding what to do next; it's following a scripted sequence.

This is the right design choice for this specific task. Phase gating is a well-defined, repeatable business process. Workflows offer exactly what this needs: predictability and consistency. An agent would introduce unnecessary complexity and risk—what if it decided to skip a gate check or approve a phase that hadn't met its criteria?

What Would It Take to Become an Agent?

If I wanted to turn this workflow into a genuine agent, I would need to make three fundamental changes:

First, introduce model-driven decision-making. Instead of a fixed path (request → validate → update), the LLM would need to evaluate the current state and choose its next action. For example, if checklist validation fails, an agent might decide to: (a) notify the project manager with specific remediation steps, (b) automatically escalate to a QA reviewer, or (c) query additional context before making a decision. The LLM would direct its own process, not follow a predetermined code path.

Second, use MCP tools for external actions. Through MCP, the agent could call tools to: query the checklist database, send notifications, or trigger external workflows—all based on its own reasoning about what's needed in each unique situation.

Third, implement a feedback loop. A true agent doesn't just execute; it learns from outcomes. After each phase advance decision, the agent would evaluate the result and adjust its future behavior. This is where the "dynamic" part of "dynamically direct their own processes" comes to life.

One Concrete Agent Upgrade

For my pipeline, the most valuable agent upgrade would be an **intelligent gate validator**. Instead of simply checking a binary checklist, an agent could:

- Read the checklist items via MCP resources
- Evaluate the quality of each item against historical patterns
- Use MCP tools to request clarifications or additional evidence
- Make a nuanced recommendation: approve, reject, or flag for human review

This would transform a rigid gate check into an adaptive quality assessment, handling edge cases that a hardcoded workflow would miss—while still maintaining human oversight for critical decisions.

Evidence of MCP Working: The attached screenshots show my Figma-designed dashboard and the completed "Manage Workflow/Phase Progress" DFD. The Figma integration demonstrates one working MCP connection—Claude accessed and reasoned about design files that chat alone could not interpret. The rate-limit diagnosis and mobile-responsive layout work were also performed through MCP-connected tool calls, not plain chat.